# CprE 3810: Computer Organization and Assembly-Level Programming

# Project Part 1 Report

Team Members: _____Preston Gaskill_____

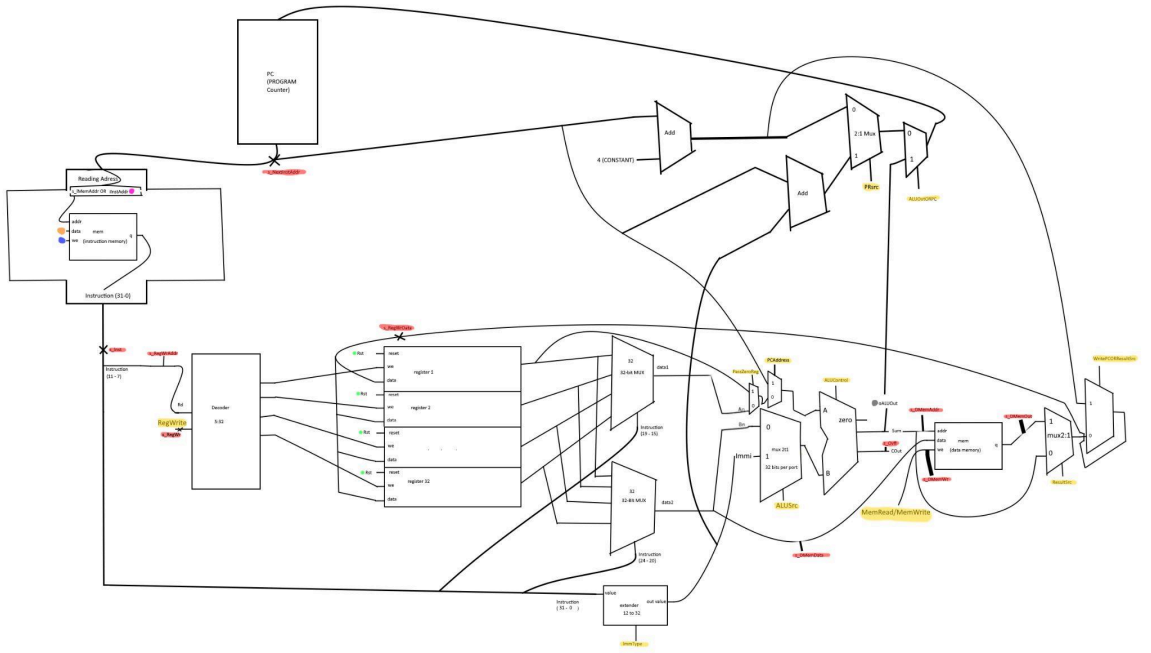
_____N/A_____

_____

Project Teams Group #:_____N/A_____

***Refer to the highlighted language in the project 1 instruction for the context of the following questions.***

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.

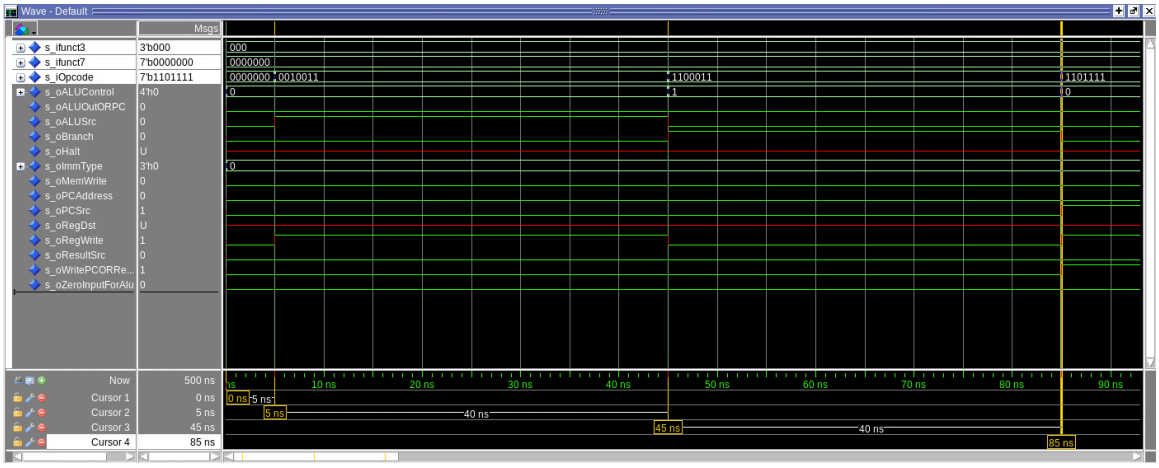



[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The end result should be an

*N*M* table where each row corresponds to the output of the control logic module for a given instruction.

Attached in a different file

[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).




There are three test cases here; all of our signals for these test cases should be set to 0 unless otherwise noted below (ignore regdst and halt)

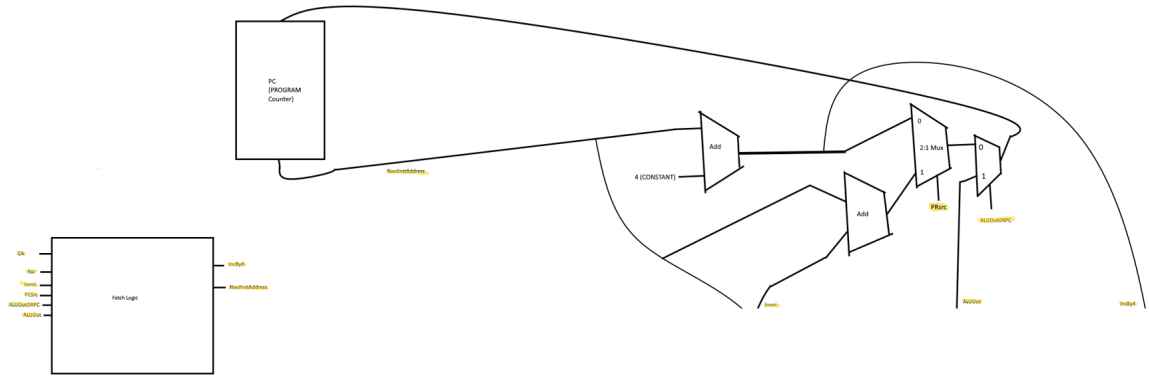
**Addi [iopcode = 0010011, funct3 = 000, funct7 = 0000000]**
ALUSrc = 1
RegWrite = 1
—-
**Beq [iopcode = 1100011]**
Branch = 1
ALUControl = 0001
—-
**Jal [iopcode = 1101111]**
RegWrite = 1
WritePCORResultSrc = 1
PCSrc = 1

[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

1. We have our standard incrementation for fetching the next instruction [PC = PC + 4]
2. For branching, we also must be able to add and immediate value to our PC [PC = PC + immi]

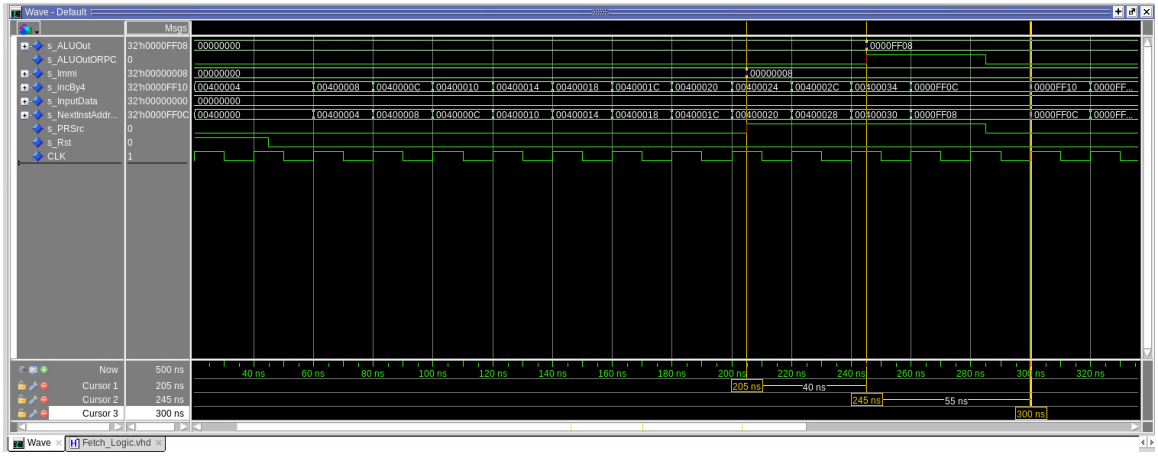
3. For jump instructions, we must be able to feed the ALU result into our PC register. [PC = ALUResult]

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?




Additional control signals: PRSrc, ALUOutORPC, Branch

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.




The fetch logic simply increments the PC register from 0x00400000 until it reaches 0x00400020. Once it reaches that value, it increments twice more by adding 0x00000008, until the value is 0x00400030. After that, it sets the value to 0x0000FF08 and then increments the PC register by 4, repeating this process indefinitely, so the next value is 0x0000FF0C and so on.

[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?

Both instructions shift the register to the right; the difference between them is whether it's a logical or arithmetic shift. This means that a logical shift would simply shift the binary digits to the left by X amount, while an arithmetic shift would shift the binary digits to the right and replicate the sign bit to the highest bit after the shift (ensuring a negative number remains negative).

Therefore, we don't need an SLA instruction because it would behave the same as an SLL instruction, because the sign bit wouldn't matter during the shift; so, we can simply use SRL instead.

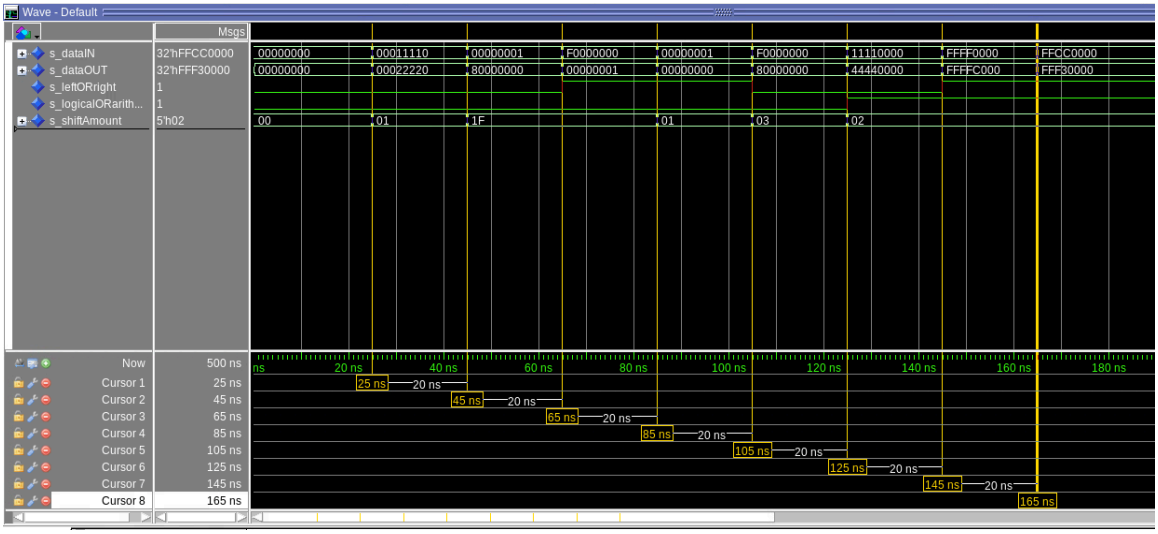
[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

The barrel shifter is built as five stages that can shift by 1, 2, 4, 8, or 16 bits. Each stage computes a shifted version of its input and then a 2t1_N mux chooses either the unshifted value or the shifted value, based on the bit of shiftAmount, so the combinations of the stages can make any shift from 0–31 bits.
All left shifts are logical shifts and will shift normally, but right shifts can be arithmetic or logical shifts, where the sign bit gets repeated when arithmetic.

[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

You can enhance the right barrel shifter by adding a direction control signal and, in each stage, conditionally wiring the shift either toward the MSB or toward the LSB.

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.




1. Set everything to 0 (logical)

    a. dataOUT = 0x00000000
2. Shift 0x00011110 to the left 1 (logical)
    a. dataOUT = 0x00022220
3. Shift 0x00000001 to the left 1 (logical)
    a. dataOUT = 0x80000000
4. Shift 0xF0000000 to the right 31 (logical)
    a. dataOUT = 0x00000000
5. Shift 0x00000001 to the right 1 (logical)
    a. dataOUT = 0x00000000
6. Shift 0xF0000000 to the left 3 (logical)
    a. dataOUT = 0x80000000
7. Shift 0x11110000 to the left 2 (arithmetic)
    a. dataOUT = 0x44440000
8. Shift 0xFFFF0000 to the right 2 (arithmetic)
    a. dataOUT = 0xFFFFC000
9. Shift 0xFFCC0000 to the right 2 (arithmetic)
    a. dataOUT = 0xFFF30000

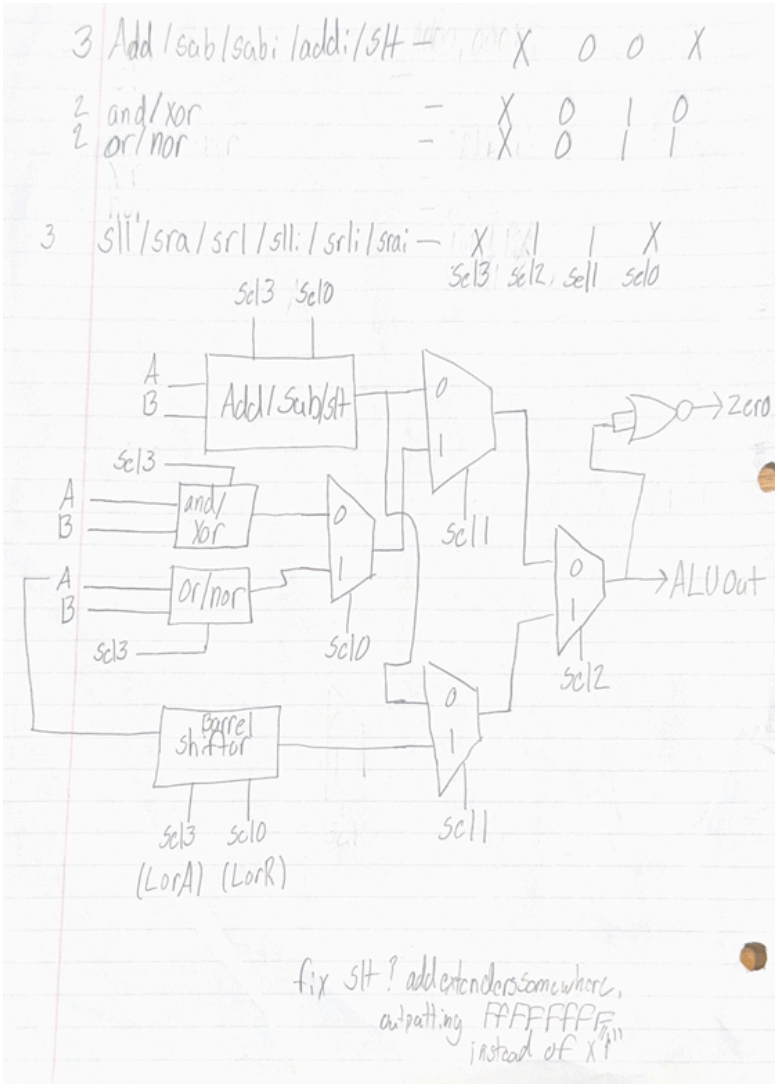
[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

My process was to identify the biggest "actions" that needed to happen within the design, and from there, it was just a matter of adding muxs and designating certain signals to ensure everything happened correctly.
One design decision I had to make was figuring out how the ALU would know when to set zero_Out to high. I was going to use an adder, but I found that an XOR gate had the game effect.

[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

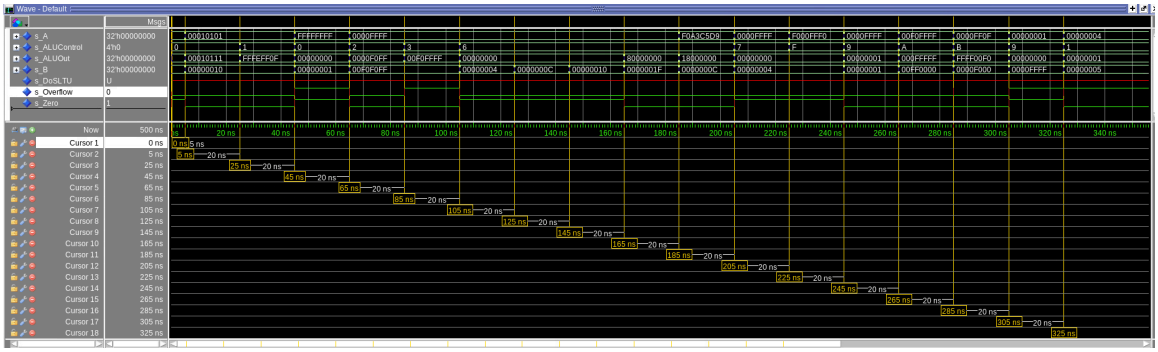
I didn't need to write any testbenches for my additional components in the ALU, as the parts I used had already been tested previously. I made some slight modifications that weren't enough to change their original functionality, but did add something to the overall components (ex., to the AdderSub, I added functionality for slt, so I just tested this in the overall ALU component).

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?




Zero is calculated by feeding ALUOut into an XOR gate, and slt is implemented by combining the add/sub/slt functions into a single component and using sel3 to determine when each function is performed.

[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.



ALUControl Guide
add/addi is 0000
sub/subi is 0001
slt is 1001

and is 0010
xor/xori is 1010

or/ori is 0011
nor is 1011

sll/slli is 0110
srl/srli is 0111
sra/srai is 1111

1. add/addi: A = (00010101), B = (00000010)
    a. Expect = 0x 00010111

2. sub/subi: A = (00010101), B = (00000010)
    a. Expect = 0x 000100F1

3. add/addi (check for overflow): A = (FFFFFFFF), B = (00000001)
    a. Expect = 0x 00000000

4. AND : A = (0000FFFF), B = (00F0F0FF)
    a. Expect = 0x 0000F0FF

5. OR: A = (0000FFFF), B = (00F0F0FF)
    a. Expect = 0x 00F0FFFF

6. sll/slli: A = (0000FFFF), B = (00000004)
    a. Expect = 0x 000FFFF0

7. sll/slli: A = (0000FFFF), B = (0000000C)
    a. Expect = 0x 0FFFF000

8. sll/slli: A = (0000FFFF), B = (00000010)
    a. Expect = 0x FFFF0000

9. sll/slli: A = (0000FFFF), B = (0000001F)
    a. Expect = 0x 00000000

10. sll/slli: A = (F0A3C5D9), B = (0000000C)
    a. Expect = 0x C5D90000

11. sll/slli: A = (0000FFFF), B = (00000004)
    a. Expect = 0x 00000FFF

12. sra/srai: A = (F000FFF0), B = (00000004)
    a. Expect = 0x FF000FFF

13. Slt (positive): A = (0000FFFF), B = (00000001)
    a. Expect = 0x 00000000

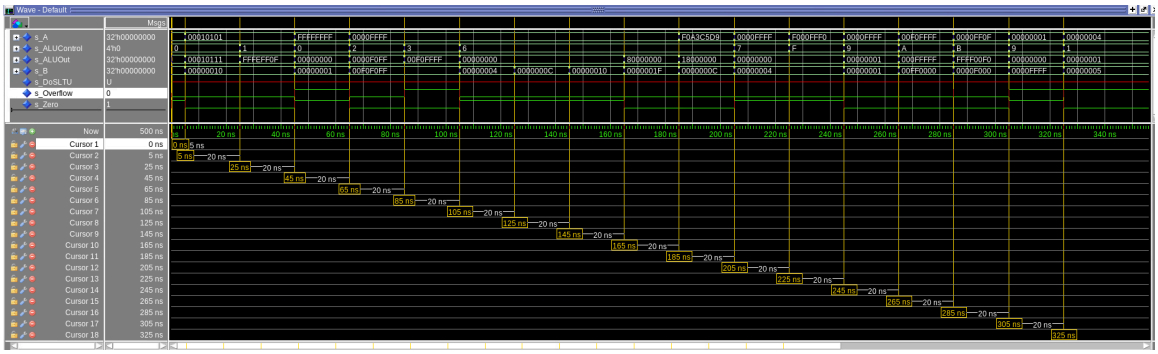
14. xor: A = (00F0FFFF), B = (00FF0000)
    a. Expect = 0x 000FFFFF

15. NOR: A = (00F0FFFF), B = (00FF0000)
    a. Expect = 0x 0000F000

16. Slt (negative): A = (000000001), B = (0000FFFF)
    a. Expect = 0x 00000001

17. Sub:  A = (000000004), B = (00000005)
    a. Expect = 0x FFFFFFFF

[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

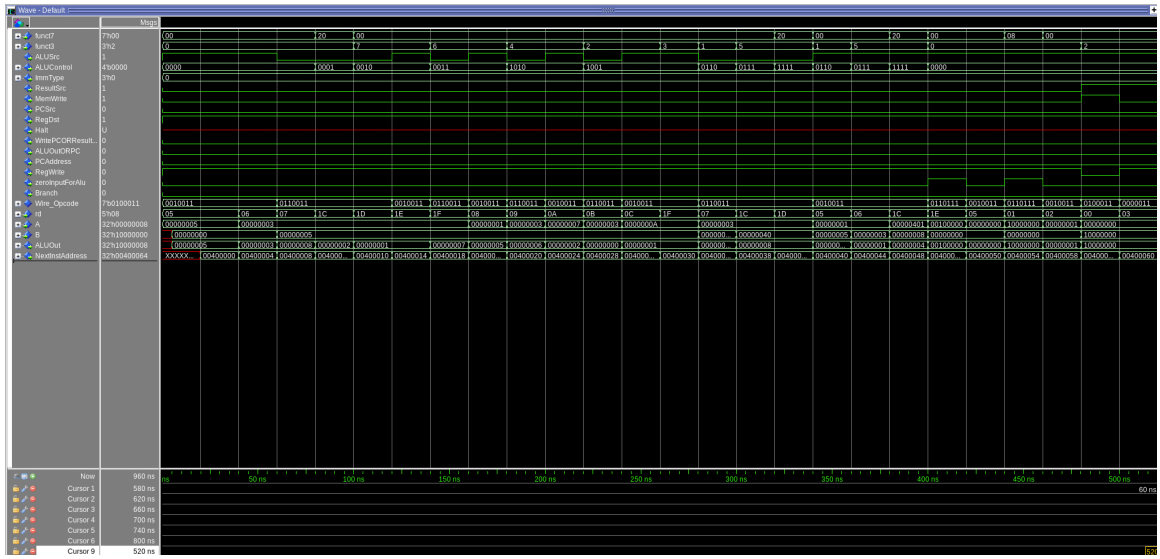


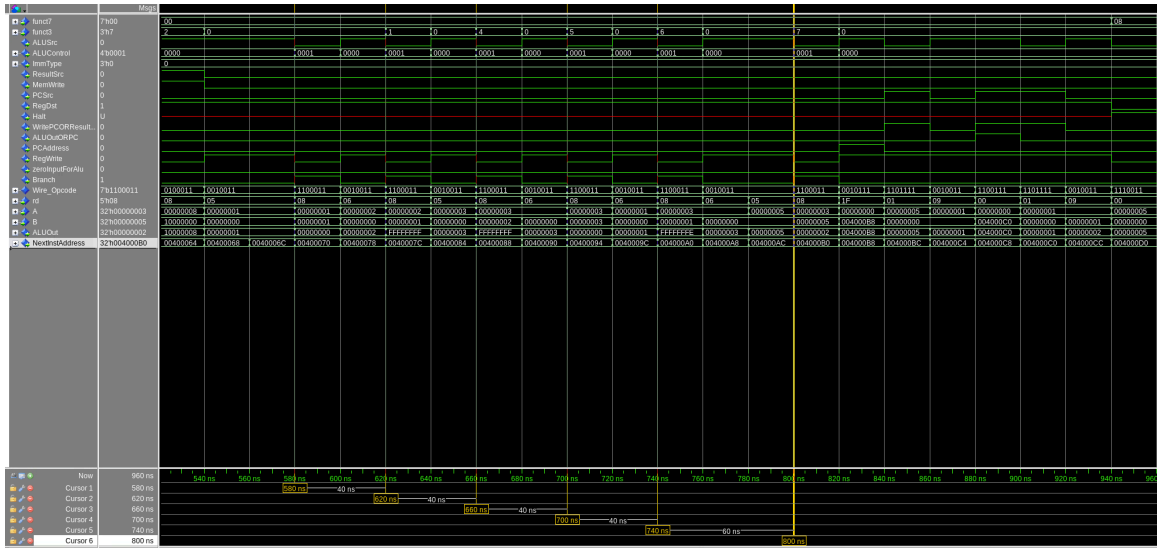
I don’t understand this question. I already tested my ALU with the correct waveforms in the previous question. My testbench is comprehensive because I tested all the functions needed from my ALU with an emphasis on the instructions that I was having trouble with (ex. slli). I also covered several edge cases that I anticipated would cause trouble, and from there, I feel it is best to debug the ALU as problems arise in my unit tests.

[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

I added the waveforms below.

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.

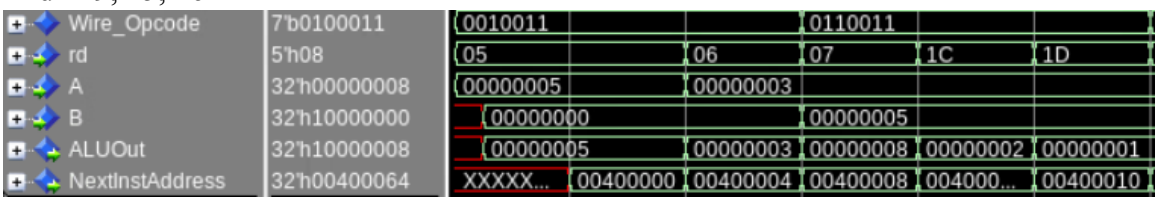




The purpose of this program was to test every instruction that the CPU supports at least once.
Given the waveform above, you can see how, as the NextInstAddress increases, the opcode changes for each new instruction, and the corresponding rd, and ALU signals ALUOut, A, and B change.

**Example**

Addi x5, x0, 5
Addi x6, x0, 3
Add x7, x5, x6
Sub x28, x5, x6
And x29, x5, x6



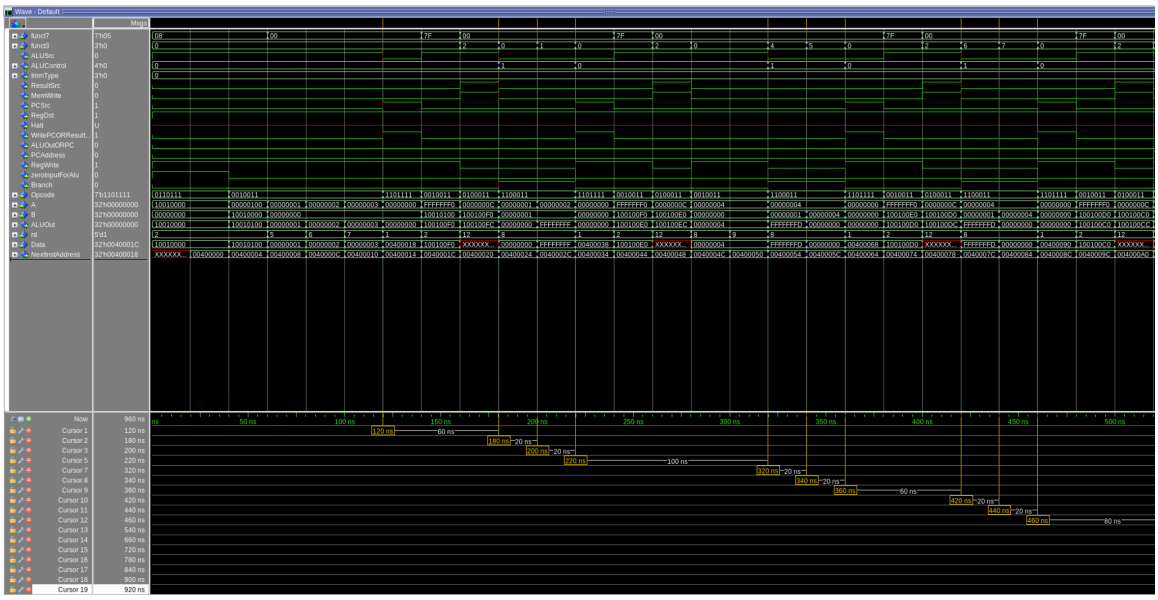

The only sections where the instructions are not executed consecutively are in our branches. I marked those locations with cursors to make them easier to see. When these branches occur, we double-check that our NextInstAddress, after the falling edge of the Branch signal, is in the correct position (it depends on the instructions between the branch and the following instruction). For example, you can see how our NextInstAddress jumped from 0x004000070 to 0x00400078 after the branch instruction was executed, and it correctly executed the addi instruction with the x6 RD instead of the addi instruction with the x7 RD.

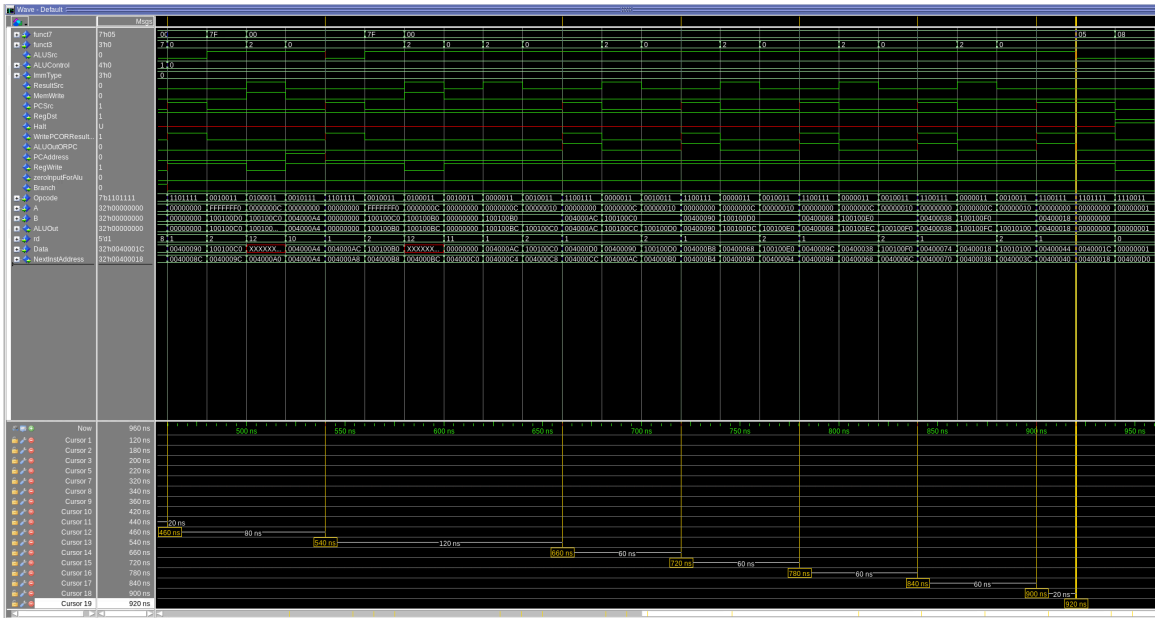
All behavior in the waveforms follows these trends until the WFI instruction is finally read and the halt instruction is set to high.

```
WARNING: Software tree location not set or invalid, using $MGC_HOME=/usr/local/mentor/calibre
All VHDL src files compiled successfully
Testing file: proj/riscv/Proj1_base_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 47
Processor Cycles: 47
CPI:   1.0
Results in: output/Proj1_base_test.s
```

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.

```
bash-5.1$ ./3810_tf.sh test proj/riscv/Proj1_cf_test.s
Using VDI Python Environment
Testing
WARNING: Software tree location not set or invalid, using $MGC_HOME=/usr/local/mentor/calibre
All VHDL src files compiled successfully
Testing file: proj/riscv/Proj1_cf_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 47
Processor Cycles: 47
CPI:   1.0
Results in: output/Proj1_cf_test.s
```





The purpose of this test was to confirm that my processor would still execute the correct instruction, even when presented with a long series of instructions that included jumps and branches.

This is the same situation as before, where we know an instruction is correct if the Opcode signal, RD, A, B, and ALUOut values match. But it's not hard to know if an instruction is correct; what's hard is knowing if the instruction is being executed at the correct time.

To help make this easier, I have included several cursors in our waveform, where each cursor marks the start of an instruction, either branching, performing a JAL, or performing a JALR. Below is a list of instructions that execute immediately before a JAL or a branch (each branch is always taken). If we see this instruction immediately before a cursor in the waveform, we know it is executing at the correct time.

**The instruction right before each JAL and branch**

1. Addi x7, x0, 3
2. Sw x1 12(x2)
3. Beq x5, x5, func1_beq
4. Bne x5, x6, func1_bne
5. Addi x9, x0, 4
6. Blt x5, x8, func2_blt
7. Bfe x9, x8, func2_bge
8. Sw x1, 12 (x2)
9. Bltu x5, x8, fun3_bltu
10. Bgeu x9, x8, func3_bgeu
11. Auipc x10, 0

After comparing this, all of our instructions should align correctly with the waveform. From this point forward, all cursors after approximately 660 ns should now mark a jalr instruction being performed. Using the same ideas as before, but a little backwards, now every instruction that we see after the jalr instruction is performed is listed below.

**The instruction right after each jalr**

1. Lw x1, 12(x2)
2. Lw x1, 12(x2)
3. Lw x1, 12(x2)
4. Lw x1, 12(x2)
5. Jal x1, end

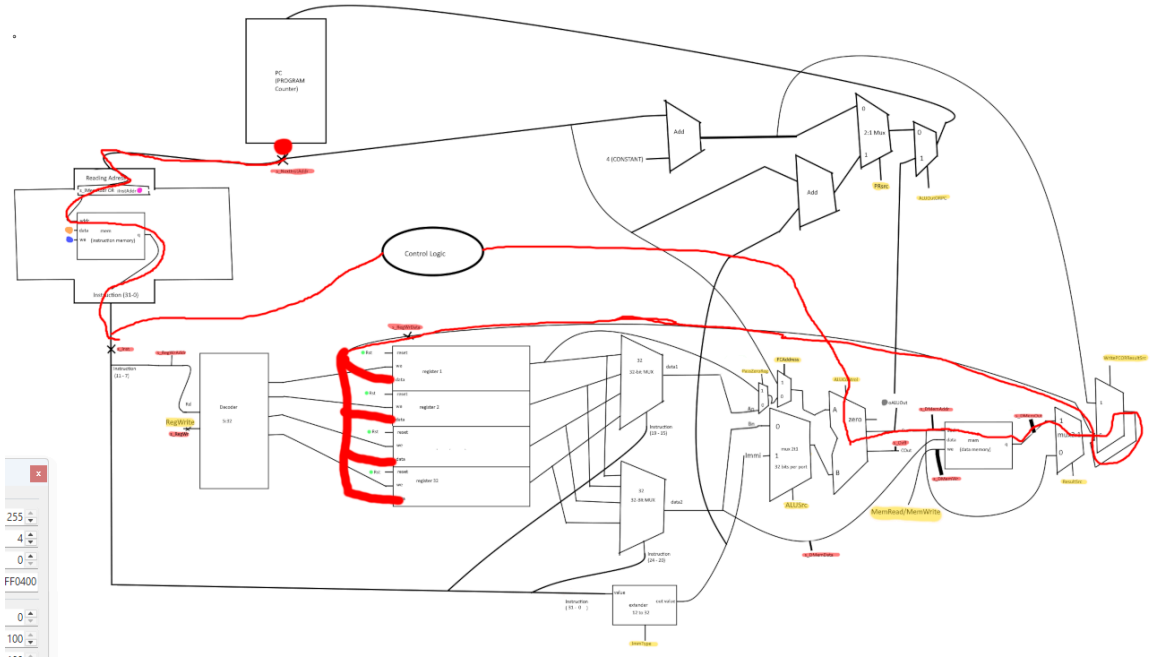
All instructions after the jalr should match the instructions above, and after the final jal, we should encounter a WFI instruction, after which our program ends, as halt is set to 1.

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.

OMIT

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

My VM locked me out, and it wouldn't let me log back in while it was running the process. I waited about two hours and a little longer because that's how long it took the first time. If the synthesis appears incorrect or incomplete, please let me know, and I will redo it.
My maximum frequency was 23.65 MHz.




I don't entirely understand how my control logic and my instruction memory are connected (maybe I'm just forgetting something), but if I were to improve this weird critical path, I would focus on improving the ALU (it has a VERY large spike for some reason), my control logic (it also has a fair spike but not as bad as the ALU), and both of my datamemories but those we're given to us to I realistically not see myself making and big changes to either of those.